

HexModX

User Manual – Groovy script

BOSCH

Abstract

This document shall help the user of HexModX to know how to work with the tool. The focus is set to enable the user to generate a modified hex file in a short time.

Table of Contents

	Abstract.....	2
	Table of Contents	3
1.	Introduction	5
1.1	General	5
1.1.1	What is HexModX?	5
1.1.2	What is XML?	5
1.1.3	What is Schema?	5
1.1.4	What is BeanShell Script?	6
2	Overview	7
2.1	Executable	7
3.	Configuration Files for HexmodX.....	7
4.	BeanShellScript for HexmodX	7
4.1	Hex Object creation.	7
4.2	Basic Input Output Operation	8
4.2.1	Parsers	8
4.2.1.1	Intel Hex Parsers.	8
4.2.1.2	Motorola S19 parsers.	8
4.2.1.3	Binary File parser	9
4.2.2	Serializers	10
4.2.2.1	Formatters	10
4.2.2.2	Serialize	11
4.3	Read, Write and Delete Operations	13
4.3.1	Read	13
4.3.2	Write	13
4.4.3	Delete	13
4.5	Memory operations	14
4.5.1	MEM-COPY	14
4.5.2	MEM-MOVE	14
4.6	Infoblock Operations	15
4.6.1	BLKLINK	15
4.6.2	BLKBUILD	16
4.6.3	WRITECKS	16
4.6.4	DETAILEDLOG	17
4.6.5	Patch or test SWTTNR	17
4.6.6	INFOTABLELOG	17
4.6.7	Patch or test SWID	18
4.7	Compatibility	18
4.8	Checksum Operations	19
4.9	SET/TEST Valid pattern	20
4.10	ABM module	20
4.11	DEF-RANGE	21
4.12	SegmentA translation	22
4.13	Variant Coding	22
4.14	Patch, Check and View operations.	23
4.15	Checksum Module	26
4.16	Exception and Logging	29
4.17	Hexcompare	30
4.18	Cleanup	31
4.19	Compress Module	32
4.20	DeleteBlock	33

1. Introduction

1.1 General

1.1.1 What is HexModX?

HexmodX is a tool to support the use cases for **MDG1** environment. The tool is designed to read and modify an Intel or S19 or any file formats that can be flashed to ECU. This modification involves reading, writing, merging and memory operations like copy, move or delete. HexmodX tool is designed for modular approach. User can invoke this tool either with XML configuration files or groovy script. (Note:HexmodX should not be used for MEDC17 environment)

1.1.2 What is XML?

XML (eXtensible Markup Language) is a simple, very flexible text format derived from SGML. XML is a markup language for documents containing structured information. XML is mainly used for exchange of a wide variety of data between two parties.

To define the structure of XML-based configuration file, users need a “Schema” or a “DTD (Document Type Definition)”. In the header of each XML this schema name should be mentioned for validation purposes.

1.1.3 What is Schema?

“Schema” is a definition language used by XML for describing and constraining the content of XML documents. The “Schema” defines the grammar and structure of the document. User can define the structure, constraints and data types in Schema.

Relative Schema path:

Sample xml file for Schema Validation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1 .\schemas\rules.xsd">
  <HEXMOD>
    <MAINMOD>
      <LOADDLL>
        <DLL NAME="BaseIO" CFG="Input.xml"/>
        <DLL NAME="BaseIO" CFG="Output.xml"/>
      </LOADDLL>
    </MAINMOD>
  </HEXMOD>
</CONF>
```

User must provide a space between www.rbin.com/mds-1 and [.\schemas\rules.xsd](#) in [xsi:schemaLocation](#) attribute. If the space is not provided, Hexmod will throw an error. User must provide single dot (.) in “[.\schemas\rules.xsd](#)” then Hexmod tries to load the rules.xsd from the schema folder inside the hexmod installation directory.

Configurable Schema path:

Sample xml file for Schema Validation is as follows.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="www.rbin.com/mds-1 C:\schemas\rules.xsd">
  <HEXMOD>
  <MAINMOD>
  <LOADDLL>
    <DLL NAME="BaseIO" CFG="Input.xml"/>
    <DLL NAME="BaseIO" CFG="Output.xml"/>
  </LOADDLL>
  </MAINMOD>
</HEXMOD>
</CONF>
```

User must provide a space between www.rbin.com/mds-1 and C:\schemas\rules.xsd in **xsi:schemaLocation** attribute. If the space is not provided, Hexmod will throw an error. If the rules.xsd is not present in "C:\schemas\rules.xsd" then Hexmod tries to load the rules.xsd from the schema folder inside the hexmod installation directory.

1.1.4 What is BeanShell Script?

BeanShell is a small, free, embeddable Java source interpreter with object scripting language features, written in Java. BeanShell dynamically executes standard Java syntax and extends it with common scripting conveniences such as loose types, commands, and method closures like those in Perl and JavaScript. All the operations like read, write, merge, move, copy and delete can be performed by HexmodX by writing commands in the beanshell script which is explained in detail in [Chapter 4](#).

2 Overview

2.1 Executable

HexModX is a command line application which takes xml configuration files or BeanShellScript as command line input. The set of operations to be performed by the HexmodX is defined in these input xml files or the beanshell script file.

Calling HexModX from dos prompt using configuration file

Open the command prompt by doing following steps
Click Start button in windows → choose Run → type cmd → click OK
Command prompt opens
Type HexMod along with configuration file as shown below

```
C:\WINNT\system32\cmd.exe
C:\>HexmodX /G:\HEXMODX\Samples\rules.xml
```

Calling HexmodX from dos prompt using BeanShellScript

```
C:\WINNT\system32\cmd.exe
C:\>HexmodX C:\HEXMODX\Samples\hexmodBSscript.bsh
```

3. Configuration Files for HexmodX.

In order to integrate the tool in build environment or in the current existing environment , HexmodX can be called using the xml configuration files. There is support provided for rules.xml, input.xml, infoblock.xml, patchcheck.xml and output.xml. Kindly refer the user manual of Hexmod18 for more information regarding the configuration files for executing the tool.

4. BeanShellScript for HexmodX

4.1 Hex Object creation.

A hex object is created initially. The hexobject defines various operations that can be performed on the hex data model. Creating a hex object results in creating a hex data model that is populated by parsing a given input hex or s19 or any known format files.

A hex object can be created using the below script.

```
IHexDataInterface hexObject = HexObjectFactory.createHexObject();
```

The following packages need to be imported for the above operation.

```
import com.bosch.hexmodx.HexInterfaces.IHexDataInterface;  
import com.bosch.hexmodx.HexInterfaces.HexObjectFactory;
```

4.2 Basic Input Output Operation

4.2.1 Parsers

Parsers are used to parse the input hex files. Parsers convert the hex file into a hex object which can be manipulated by the tool. There are two types of parsers which can be used in the beanshell script.

- IntelHexParser
- MotoS19 Parser

4.2.1.1 Intel Hex Parsers.

Intel hex parsers are used to parse hexfiles which are of Intel Format. The script to be written is as follows:

```
HexmodxFileParser parser = new IntelHexFileParser();  
parser.parse(hexObject, "input.hex" , true);
```

The function parse takes three parameters as the input.

The first parameter is the hexobject, the second is the name of the input hex file. If the input hex file is present in the same location as that of the beanshell script, only the name of the input hex file can be given. Else the complete path of the input hex file has to be given.

```
parser.parse(hexObject, "C:\\HEXMOD\\Sample\\input.hex" ,true);
```

The third parameter is used to indicate “**OVERWRITE**”, which is a Boolean attribute and it is mandatory. The significance of this attribute is particularly during merge operation, which is explained in detail in section 3.3.

The following package needs to be imported for the above operation.

```
import com.bosch.hexmodx.parsers.IntelHexFileParser;
```

4.2.1.2 Motorola S19 parsers.

Motorola S19 parsers are used to parse hex files which are of S19 Format.
The script to be written is as follows:

```
HexmodxFileParser parser = new MotorlaS19Parser();  
  
parser.parse(hexObject, "input.s19" , true);
```

The function parse takes three parameters as the input.

The first parameter is the hexobject, second is the name of the input S19 file. If the input S19 file is present in the same location as that of the beanshell script, only the name of the input file can be given. Else the complete path of the input hex file has to be given.

```
hexobject.parse(hexObject , "C:\\HEXMOD\\Sample\\SAMPLE2.S19", true);
```

The third parameter is used to indicate “**OVERWRITE**”, which is a Boolean attribute and it is mandatory. If you are parsing a single S19 file, the value of this attribute is not significant. The significance comes into picture in case of multiple parse command. This section is explained in detail in section 3.3.

The following package needs to be imported for the above operation.

```
import com.bosch.hexmodx.parsers.MotorlaS19Parser;
```

4.2.1.3 Binary File parser

Binary File parser is used to parse binary files which are of Binary Format.
The script to be written is as follows:

```
BinaryFileParser binFileParser = new BinaryFileParser();  
binFileParser.parse(hexObject, "input.bin", true);
```

The function parse takes three parameters as the input.

The first parameter is the hexobject, the second is the name of the input bin file. If the input bin file is present in the same location as that of the beanshell script, only the name of the input hex file can be given. Else the complete path of the input hex file has to be given.

```
binFileParser.parse(hexObject, "C:\\HEXMOD\\Sample\\input.bin" , true);
```

The third parameter is used to indicate “**OVERWRITE**”, which is a Boolean attribute and it is mandatory. The significance of this attribute is particularly during merge operation, which is explained in detail in section 3.3.

The following package needs to be imported for the above operation.

```
import com.bosch.hexmodx.parsers.BinaryFileParser;
```

Ignore Line Checksum

This is an optional boolean attribute with default value “false”
This attribute provides HexModX application to ignore the wrong line checksums while reading the input Hex files. If the this command is set to “true”, then HexMod will issue a WARNING message for the wrong line checksum found.

Application will throw ERROR and abort if this option is not set.

To set this attribute the below API can be used on parser.

```
parser.setIgnoreLineCks(boolean value);
```

4.2.2 Serializers

Serializers are used to generate the output hex file. Serializers serialize the hexobject into a format, specified by the user and write it into an output file.

HexmodX provides a generic serializer which can serialize the object into any type of hex file based on the formatter provided. Serializer is generic because of introduction of formatters for serialization.

4.2.2.1 Formatters

Formatter transforms the hex object into a specific format based on the user command.

There are two types of formatters provided by HexmodX.

- Intel hex formatter.
- Motorola hex formatter
- Binary File formatter

Based on the required output file type, respective formatters can be used.

Intel Hex formatter

Intel hex formatter is used if the output file has to be in the standard Intel hex format. The script used is:

```
IFormatter formatter= new IntelFormatter();
```

This formatter created is passed as an argument for serialize.

The package to be imported is

```
import com.bosch.hexmodx.formatters.IntelFormatter;
```

Motorola hex formatter

Motorola hex formatter is used if the output file has to be in the S19 hex format

The script used is

```
IFormatter formatter = new MotorolaS19Formatter();
```

This formatter created is passed as an argument for serialize.

We can also specify the format of S19 file.

In case the file has to be written in S2 format, it can specified using the below script.

```
formatter.setFormat("S2");
```

“S3” is taken as the default format if the above script is not specified or the argument is mentioned as “null”.

The package to be imported is

```
import com.bosch.hexmodx.formatters.MotorolaS19Formatter;
```

Binary File formatter

Binary File formatter is used if the output file has to be in the bin format
The script used is

```
IFormatter formatter = new BinaryFileFormatter();
```

This formatter created is passed as an argument for serialize.

The package to be imported is

```
import com.bosch.hexmodx.formatters.BinaryFileFormatter;
```

4.2.2.2 Serialize

Once the formatter is created, we can use the serialize command to generate the output file. We need to create a object of HexmodxSerializer and pass the hex object and the formatter (which is created in the section 4.2.2.1) as the arguments in the constructor. The script for creating the new object is as follows.

```
HexmodxSerializer hexFileSerializer = new HexmodxSerializer(hexObject,
Formatter);
```

If the formatter is not specified, Hexmodx would abort throwing an error.

The output file name can be specified in the serialize function.

```
hexFileSerializer.serialize("output.hex");
```

The output file will be generated in the same path as that of the beanshellscript. The long path can also be specified.

```
hexFileSerializer.serialize("C:\\HEXMOD\\output.s19");
```

To serialize the binary file, create BinaryFileSerializer object and invoke serialize method.

```
HexmodxSerializer hexFileSerializer = new BinaryFileSerializer(hexObject,
formatter);
```

```
hexFileSerializer.serialize("output.bin");
```

Specifying the layout

User can specify the layout in which the output hex file should be. It is optional. In case the user doesn't specify the layout, the layout of the output hex file will be 0x20 by default.

```
hexFileSerializer.serialize("output.hex", (byte) 0x10);
```

In the above script (byte)0x10 indicates the layout of the output file.

Serializing for a particular range

Serialize also supports the feature, which can be utilized to merge several hex range data into single hex file.

User can specify the range by creating a instance of IRange or list of IRange objects.

```
IRange range = new Range(long startAddress, long len);

hexFileSerializer.serialize("output.hex", range);

hexFileSerializer.serialize("output.hex", range, layout);
```

For more than one range a list of IRange objects can be passed.

```
List<IRange> rangeObjects = new ArrayList<IRange>();
IRange range1 = new Range(long startAddress, long len);
IRange range2 = new Range(long startAddress, long len);
rangeObjects.add(range1);
rangeObjects.add(range2);

hexFileSerializer.serialize("output.hex", rangeObjects);

hexFileSerializer.serialize("output.hex", rangeObjects, layout);
```

Merge Operation

HexmodX can be used to merge two or more files. Merging of hex files takes place if there is more than one parse command. Initially the parser needs to be created as discussed in 4.2.1 based in the type of hex file and then if multiple parse commands are given, merge automatically occurs.

```
hexObject.parse("input1.hex", true);
hexObject.parse("input2.hex", true);
```

In the above example, input1.hex and input2.hex will be merged. In the case of merge, "OVERWRITE", which is the second argument in parse plays a vital role.

If overwrite is made as true, any duplicate address found will be overwritten by the second file i.e if the second file has some addresses which are same as the first hex file, the data of the first hex file is overwritten by the second file.

If overwrite is made as false, if there is a duplicate address in the second hex file, the tool aborts i.e if the second file has any addresses which is same as the first hex file, the tool throws an error and aborts.

Merge feature can be used to merge:

- Two or more intel hex files
- Two or more S19 files.
- Combination of Intel and S19 files.

4.3 Read, Write and Delete Operations

HexmodX can be used to perform basic read, write or delete operation on a particular address as specified by the user.

4.3.1 Read

Read operation can be performed to read the data at a particular address. The length of the data to be read should also be specified. There are two functions available for the user. They are:

```
read(Object address, int len)
read(Object address, int len, String fillGapValue)
```

The below script is used to perform read operation. To see the value on the output window, sys out command can be used. Read function has a return type of byte[].

```
byte[] data = hexObject.read("0x80020120", 8);
```

In case there is a gap in between, the gaps are indicated by the value "F3". User can also specify the Fill Gap value, which is an optional attribute. The value has to be a char value indicated in single quotes.

```
byte[] data = hexObject.read("0x80020120", 8, 'F');
```

Fill Gap value is an optional attribute.

4.3.2 Write

Write operation can be performed to write the data user wants at a particular address. There are two functions available. User can use any of these based on the requirement.

```
write(Object address, byte[] data, boolean overWrite);
write(Object address, long, byte[] data, boolean overWrite);
```

Address indicates the address to which the data has to be written.

Data indicates the data to be written.

Len indicates the length of bytes to be written.

OverWrite is used to enable or disable overwriting of the address.

If overwrite is made as true, if there is any data present in the destination address, it will be overwritten.

If overwrite is made as false, if there is any data present in the destination address, the tool aborts.

4.4.3 Delete

Delete operation can be performed to delete certain length of data at a particular address. The script to do the delete operation is as follows.

```
deleteData(Object address, long len);
```

The first attribute indicates the address, where the delete operation has to be performed. The second attribute indicates the length of the data to be deleted.

```
hexObject.deleteData("0x8002FFFF",10);
```

4.4.4.5 Memory operations

There are two types of memory operations, which can be performed by the tool. They are:

- MEM-COPY
- MEM-MOVE

4.5.1 MEM-COPY

MEM-COPY is used to copy certain length of memory from one address to another address specified by the user within the same hex data model. Mem-copy creates a copy of data from one address to another address.

```
MemCopy(Object fromAddress, long len, Object toAddress, boolean overWrite, String fillgaps)
```

fromAddress indicates the address from which the data has to be copied.

Len indicates the length of bytes to be copied from the source address.

toAddress is the destination address, the address to where the data has to be copied to.

overWrite is used to enable or disable overwriting of the address.

Fillgaps is used as fill gap value, in case there is gap present in the data to be copied.

If overwrite is made as true, if there is any data present in the "to" address will be overwritten. If overwrite is made as false, if there is any data present in the "to" address, the tool aborts.

A sample script is shown below.

```
hexObject.MemCopy("0x70000000", 0xFFFF, "0x80010000", true, "00");
```

4.5.2 MEM-MOVE

MEM-MOVE is used to move certain length of memory from one address to another address specified by the user within the same hex data model. Mem-Move completely moves the chunk of data from one address to another address specified by the user.

```
public void MemMove(Object fromAddress, long len, Object toAddress, boolean overWrite, String fillgaps)
```

fromAddress indicates the address from which the data has to be moved.
Len indicates the length of bytes to be moved from the source address.
toAddress is the destination address, the address to where the data has to be moved to.
overWrite is used to enable or disable overwriting of the address.
Fillgaps is used as fill gap value, in case there is gap present in the data which is to be moved.

If overwrite is made as true, if there is any data present in the "to" address will be overwritten.
If overwrite is made as false, if there is any data present in the "to" address, the tool aborts.

A sample script is shown below.

```
hexObject.MemMove("0x70000000", 0xFFFF, "0x80010000", true, "00");
```

4.6 Infoblock Operations

Infoblock module is required to perform the modification of infoblocks inside EDC18 software. This Infoblock module performs linking and building of each blocks, modify blocks for checksum, compatibility etc.

InfoBlockManager is the interface which provides the BLKLINK, BLKBUILD and checksum functionalities (WRITECKS) to the user. The object of InfoblockManager can be created as shown below:

```
IInfoblockManager infoBlockManager =
AbstractInfoblockManagerFactory.createInstance(Object
infoBlockAddress, IHexDataInterface hexDataModel);
```

The following package has to be imported:

```
import com.bosch.hexmodx.mdg1.infoblock.InfoblockManager;
import
com.bosch.hexmodx.mdg1.infoblock.core.AbstractInfoblockManagerFactory;
```

Please note that Hexmodx support infoblock version 5 as well as version 7. Version handling will be taken care internally based on the infoblock address provided during creating the instance of infoblock.

4.6.1 BLKLINK

Linking the infoblock depends on the addresses provided in the input parameter of **BLKLINK** function. Linking Infoblock depends on the order of the blocks specified in the input parameter list. The input can be in the form of a list or just a series of strings which contains the infoblock addresses.

```
public void blkLink(List list);
public void blkLink(Object ...objects);
```

The input can be in the form of a list.
For example:

```
List blkLinkList=new ArrayList();
```

```
blkLinkList.add("0x80020000");
blkLinkList.add("0x80100000");
blkLinkList.add("0x80200000");
blkLinkList.add("0x80300000");
blkLinkList.add("0");
```

```
infoBlockManager.blkLink(blkLinkList);
```

4.6.2 BLKBUILD

BLKBUILD is used to build the already linked infoblocks. User has to give the start address of infoblock to start building the infoblock chain. The start address will be usually the start address of the Start Up block (SB).

```
public void blkBuild(Object startAddress);
```

For ex :

```
infoBlockManager.blkBuild("0x80020000");
```

Infoblock operations can also be performed using xml configuration files. Kindly refer infoblock module of hexmod18 user manual for more details.

4.6.3 WRITECKS

Balancing Infoblock is done by updating Checksum. These values can be tested or modified depending on the values TEST/SET. The user can either set or test the checksum value. These functionalities are equivalent to the WRITECKS attribute in the xml configuration file.

`updateChecksums()` is used to trigger the checksum functionality for all Infoblock and Checksum Infoblock. Checksum calculation will be done based on the ascending order of end addresses of each Checksum Infoblock.

The function can be called as shown below.

```
infoBlockManager.updateChecksums();
```

`testChecksums()` is used to test if the checksum of the infoblock and checksum infoblock is same as the checksum calculated. In case the value is not same, the tool throws an inconsistency error and aborts.

The function can be called as shown below.

```
infoBlockManager.testChecksums();
```


Please note that the above functions can be called only after BLKLINK or BLKBUILD is performed.

4.6.4 DETAILEDLOG

Hexmod tool provides an easy way for a user to write the addresses and the contents of all the basic elements of an Infoblock from a hex file to the xml file. Basic elements of Infoblock mean all the header information about the Infoblock, which will be present in the BASIS INFOBLOCK section of a block. Along with basis Infoblock, even the information about the checksum blocks and the epilogue contents and the respective addresses of the epilogue elements will also be print. The xml file which contains all these addresses and contents is generally termed as ADDRESS LAYOUT MODEL. To achieve this, a command named DETAILEDLOG will have to be mentioned in the Infoblock.xml file.

`IDetailedLogFileSerializer` interface provides the methods `serialize(List<IBlockElement> blockElements)` and `serializeSingleBlock(final IBlockElement blockElement)` to serialize the ADDRESS LAYOUT MODEL for all the infoblocks or specific Infoblock in hex file respectively.

Sample HexmodX script file:-

```
IDetailedLogFileSerializer detailedLogFile = new DetailedLog("AddressModel.xml");
```

```
List<IBlockElement> infoBlocks = infoblockManager.getInfoBlocks();
if ((infoBlocks != null) && !infoBlocks.isEmpty()) {
    detailedLogFile.serialize(infoBlocks);
}
```

4.6.5 Patch or test SWTTNR

HexmodX supports patching SWTTNR value or testing the existing SWTTNR value in hex file.

To patch the value, HexmodX invokes the below method,

```
public void setxTTNr(final byte[] xTTNr) throws Exception
```

To test the value, HexmodX invokes,

```
public byte[] getxTTNr() throws Exception
```

Sample HexmodX script file:-

```
byte[] swttnrVal = HexUtils.getBytes("0xAF AF AF AF AF AF AF AF AF AF");
```

```
List<IBlockElement> infoBlocks = infoblockManager.getInfoBlocks();
for (IBlockElement blocks : infoBlocks) {
    ((IMDGInfoblock) blocks).setxTTNr(swttnrVal);
}
```

4.6.6 INFOTABLELOG

Hexmod tool provides an easy way for a user to write the addresses and the contents of Infotable of an Infoblock from a hex file to the xml file. Each Infoblock contains an array of Infotables. Infotable array

of all infoblocks are serialized to log file. To achieve this, a command named INFOTABLELOG will have to be mentioned in the Infoblock.xml file.

`ILogFileSerializer` interface provides the method `serialize(List<IBlockElement> blockElements)` to serialize the Infotable contents of all the infoblocks in hex file.

Sample HexmodX script file:-

```
ILogFileSerializer infotableLogFile = new InfotableLog("InfotableLog.xml");
```

```
List<IBlockElement> infoBlocks = infoblockManager.getInfoBlocks();
if ((infoBlocks != null) && !infoBlocks.isEmpty()) {
    infotableLogFile.serialize(infoBlocks);
}
```

4.6.7 Patch or test SWID

HexmodX supports patching SWID value or testing the existing SWID value in hex file.

Loop through infotable arrays of all infoblocks, check if the content identifier is 0x002F i.e. SWID and patch the value into address given in infotable.

```
byte[] swidVal = HexUtils.getBytes(0x1234567801020304);
byte numInfoTab = ((IMDGInfoblock) infoblock).getNumInfoTab();
if (numInfoTab > 0)
{
    int adInfoTab = (int) ((IMDGInfoblock) infoblock).getAdInfoTab();
    // loop through infotable arrays.
    for (int j = 0; j < numInfoTab; ++j)
    {
        // Check if ContentIdentification is SWID.
        if (HexUtils.getDShort(this.hexObject.read(adInfoTab, 0x2),
this.hexObject.isLittleEndian()) == 0x002F)
        {
            // Patching SWID value into hex file
            this.hexObject.write(HexUtils.getDWord(this.hexObject.read(adInfoTab + 8,
0x4), this.hexObject.isLittleEndian()), swidVal, true);
            break;
        }
        adInfoTab += 12;
    }
}
```

4.7 Compatibility

Compatibility is the concept through which a certain degree of integrity is maintained between the infoblocks. The compatibility concept enables blocks to be tight coupled with each other i.e a level of dependency is set between the two infoblocks such that they cannot work individually. Setting of compatibility involves copying the compatibility value of one block to the other. The compatibility value is a checksum value. The ecu performs a check to validate whether the two blocks are compatible by comparing the compatibility checksum values stored in the blocks

The infoblockManager in hexmodX is provided with the APIs `setCompatibility`

```

void setCompatibility(Object masterBlkAddr, Object childBlkAddr)
public void testCompatibility(Object masterBlkAddr, Object childBlkAddr)
public void deactivateCompatibility()

```

The method setCompatibility is used to update set the compatibility between two blocks. The compatibility all the blocks that are dependent on the master block or the child block are updated accordingly.

The method testCompatibility is used to test whether given blocks are compatible with each other. This is performed by testing whether the adBlkId of block A and the adBlkIdRef of the block B are equal and also the compId of block A and the compIdRef of the block B are equal.

If any of these two conditions are not equal then the compatibility test fails and an error message is issued. And hexmodX continues with the compatibility test with the other blocks that are specified if any.

The method deactivateCompatibility is used to remove any existing compatibility between the blocks. As part of the deactivateCompatibility the adCompIdRef of all the blocks are made 0x0 such that there is no compatibility between any blocks

4.8 Checksum Operations

Checksum modules are used to calculate the checksum over the range specified. It also takes the initial and final values for Checksum calculations. Checksum calculation depends on the chosen algorithm. This can be provided by Checksum algorithm names. Possible types of algorithms supported by HexmodX currently are {CRC32, ADD32}

This checksum module is loaded by Infoblock module for the checksum over Infoblock and Checksum Infoblock. Default **CRC32** checksum algorithm will be used for the Checksum over Infoblock.

For checksum over the checksum blocks, depending on the index in the hex file (checksum Infoblock structure) the algorithm, respective algorithm will be invoked. The index for the selection of the algorithm is as shown below.

Index	Algorithm
0x1	ADD32
0x2	CRC32

Checksum object can be created with the help of checksum factory.

```

IChecksum checksumObject =
ChecksumFactory.createChecksumObject(Algorithm.CRC32, hexObject);

```

Or

```
IChecksum checksumObject =
ChecksumFactory.createChecksumObject(Algorithm.ADD32, hexObject);
```

The following packages has to be imported:

```
import com.bosch.hexmodx.HexInterfaces.ChecksumFactory;

import com.bosch.hexmodx.HexInterfaces.IChecksum;
```

The user can calculate checksums using the following functions:

```
calculateChecksum(databytes);
calculateChecksum();
```

The user can also set the start address, end address, start value and end values by using the following functions:

```
public int getFromAddress();
public void setFromAddress(int fromAddress);
public long getLen();
public void setLen(long len);
public long getStartValue();
public void setStartValue(long startValue);
public long getEndValue();
public void setEndValue(long endValue);
public boolean isAlignDataTo4Bytes();
public void setAlignDataTo4Bytes(boolean alignDataTo4Bytes);
public boolean isInverse();
public void setInverse(boolean isInverse);
public void setBigEndianConversion(boolean isBigEndianConversion);
```

4.9 SET/TEST Valid pattern

All the infoblocks in MDG1 platform contains the valid pattern in the beginning of the block. Sometimes it is needed to check and set the value of the valid pattern in the infoblock.

The methods

```
public void setValidPattern();
public boolean testValidPattern();
```

of the infoblock manager are used to set and check for the valid pattern in the beginning of the infoblocks.

setValidPattern will update the validPattern value for each and every infoblock and the testValidPattern method will check whether all the infoblocks have the validpattern value at the start address of the infoblock.

4.10 ABM module

ABM module is used to validate and trim the checksum values for the Alternate Boot Mode (ABM) area in the Hex File.

The user has to create an object of ABM using the ABM factory as shown below:

```
IABM abmObject = AbmFactory.createInstance(hexObject, HexPlatform.MDG1);
```

The following packages has to be imported:

```
import com.bosch.hexmodx.HexInterfaces.AbmFactory;

import com.bosch.hexmodx.HexInterfaces.IABM;
```

The primary address of the ABM module can be given as shown below:

```
abmObject.build("0x8002FFFF");
```

4.11 DEF-RANGE

DEF-RANGE defines the valid RANGE which should be considered while processing. DEF-RANGE contains START, LEN, FILLPATTERN and FILLGAP attributes. START and LEN defines the RANGE of the valid region and FILLPATTERN specify the values to be considered for GAPS in the defined RANGE. FILLGAP is a Boolean attribute, if set to “true”, whole defined RANGE shall be filled with values specified by FILLPATTERN. By default, this value will be “false”.

Setting of DEF-RANGE:

```
DefRange defRangeObj = new DefRange(long startAddress, long len, String fillPattern,boolean fillGaps);
```

defRangeObj can be set for the data model using the API,

```
hexObject.addDefRangeObject(DefRange defRangeObject);
```

Any number of DEF-RANGE objects can be added to a particular hexobject. Once the user has provided the DEF-RANGE, he should also set the def range present tag using the API,

```
hexObject.setDefRangePresent(boolean isDefRangePresent);
```

Once the DEF-RANGE is set, the user can also check if a particular address range is within DEF-RANGE or not, using the below API. It returns a boolean, true indicating the address range is within DEF-RANGE and false indicating the address range is outside the DEF-RANGE.

```
hexObject.checkForDefRange(long fromAddress, long length);
```

The user has to call process Def range for processing the gaps i.e filling the gaps with fill pattern. The gaps will not be processed until the below API is called.

```
hexObject.ProcessDefRangeWithFillGaps();
```

The user can get the Def Range objects using the below API.

```
hexObject.getDefRangeObjects () ;
```

The user at any point of time can clear the def range objects by calling the below API.

```
hexObject.clearDefRange () ;
```

Once the DEF-RANGE is given, any operation on the data model is done based on this range. If the data to be processed is outside DEF-RANGE the operation is skipped.

For ex: DEF_RANGE is defined from address 0x80000000 of len 0x10.

Suppose the user calls the delete data operation from address 0x80000005 of len 0x10, since the part of the data to be deleted doesn't come under DEF-RANGE, the delete operation is skipped.

4.12 SegmentA translation.

SegmentA translation is used to bypass any operations on segmentA address to segmentt8 addresses. SegmentA addresses in the hex file are not modified instead any operations on these addresses are performed on its equivalent segment8 address. The default value of this element is true. That is this feature is enabled by default. The value of the attribute can be made false if the conversion from segmentA to segment8 is not required using the below API.

```
hexObject.setSegmentTranslate( boolean TranslateAto8 ) ;
```

4.13 Variant Coding

Forward Variant Coding

Forward Variant coding can be performed by using the processForwardVaraintCoding(variantList) API provided by IVariantCoding interface. This function takes list of variant objects as input and returns a master hexObject.

```
IVariant variantObject = VariantFactory.createInstance(infoBlkAddress, hexObject);
List<IVariant> variantList = new ArrayList<IVariant>();
variantList.add(variantObject);
```

Here infoblock address is the start address of the VDS infoblock. Variants have to be created for each file and the list of variants has to be passed for forward variant coding.

```
IVariantCoding variantCoding =
VariantCodingManager.getVaraintCodingImpl () ;
IHexDataInterface hexObject =
variantCoding.processFarwardVaraintCoding (variantList) ;
```

The obtained hexObject can be serialized into a hex file. Kindly refer [section 4.2.2.2](#)

The various logging options available are:

[VDSLOG](#) which takes name of a file to which the details of the data set sizes will be written.

```
variantObject.logVdsInformation (vdsLog) ;
```

where vdsLog is the filePath of the log file to be generated

[VCTRTABLELOG](#) takes a filename as a input to which vector table contents of variant coded file is written.

```
variantObject.logVectorInfoblock(vctrInfoLog);
```

where vctrInfoLog is the filePath of the log file to be generated.

[VCTRINFOLOG](#) takes the file name as input to which the vector Infoblock table contents of variant coded file is written.

```
variantObject.logVectorTables(vctrTableLog);
```

where vctrTableLog is the filePath of the log file to be generated.

Backward Variant Coding

Backward Variant coding can be performed by using the processBackVaraintCoding(final IVariant master, final Byte fillValue) API provided by IVariantCoding interface. This function takes 2 input arguments. Master variant and the fillValue.

```
IVariantCoding variantCoding = VariantCodingManager.getVaraintCodingImpl();
IVariant variantObject =
VariantFactory.createInstance((int)HexUtils.validateAndConvert(infoBlkAddress), hexObject);
```

Here infoblock address is the start address of the VDS infoblock

```
variantList = variantCoding.processBackVaraintCoding(IVariant
variantObject,String fillValue);
```

The function processBackVaraintCoding returns a variant list. Hexobject can be obtained from each of the variant using variant.getHexDataObject();

The obtained hexObject can be serialized into a hex file. Kindly refer [section 4.2.2.2](#)

4.14 Patch, Check and View operations.

PatchAndCheck module is used to patch/check/view hex data in the hex file. Individual operations are explained as follows:

Patch operation is used for patching the hex file data with HEX or ASCII values at the specified address given in the configuration xml. Check operation is used for checking the value in the hex file with the value specified in the xml file for the specified address. View operation is used for viewing the hex content at the specified address given.

```
PatchCheck patchcheckObj = new PatchCheck(hexObject);
```

Patch Operation:

Patch operation can be performed by using the API patch provided by the IPatchCheck interface.

```
patchcheckObj.patch(final String type, long address, String
valueToBePatched, final String name, final String filePath,
int len)
```

Type can be IHEX, ASCII or REF.

Address indicates the location where the patch operation should be performed.

valueToBePatched indicates the value to be patched.

Name attribute specifies the name of the label / data to be patched.

filePath is the name of the Hex file user is patching.

Len holds the length of the data to be retrieved from hex data;

Check Operation:

Check operation can be performed by using the API check provided by the IPatchCheck interface.

```
patchcheckObj.check(final String type, long address, final String data,
final String name, final String filePath,
final int len)
```

Type can be IHEX, ASCII or REF.

Address indicates the location where the patch operation should be performed.

data indicates the value to be patched.

Name attribute specifies the name of the label / data to be patched.

filePath is the name of the Hex file user is patching.

Len holds the length of the data to be retrieved from hex data;

View Operation:

View operation can be performed by using the API view provided by the IPatchCheck interface.

```
patchcheckObj.view(final String type, long address, final int len, final
String name, final String filePath)
```

Type can be IHEX, ASCII or REF.

Address indicates the location where the patch operation should be performed.

Name attribute specifies the name of the label / data to be patched.

filePath is the name of the Hex file user is patching.

Len holds the length of the data to be retrieved from hex data;

Ref Type

Here, the patch check module will fetch the data from address given in the value field and use that data as the VALUE for the specified ACTION.

For example

```
patchcheckObj.patch("REF", 0x80008000, "80020000", "Sample", "Sample.hex",
5)
```

In this particular operation, the data to be patched is fetched from 0x80020000 of len 5, say "AABBCCDDEE" and patched at the location 80008000.

GetAddress and GetInfoTableAddress

GetAddress is used to locate an address at an offset specified from a pointer address. The address can be calculated using the API getAddress.

```
long getAddress(final long startAddress, final long offset1, final long
level)
```

For example: patchcheckObj.getAddress(0x80800024, 0x1000, 3)

Since the level is specified as 3, HexmodX will iterate the 3 times to get the final address. Final address is calculated as follows. Consider the sample hex data.

```
:0200000048080FA
:100000000000AAAAAAAAAAAAAAAAAAAA97
:1000100000000000000000000000000000E0
:100020000000000080800060000000000000D0
:1000300000000000000000000000000000C0
:1000400000000000000000000000000000B0
:1000500000000000000000000000000000A0
:10006000808000940000000000000000000090
:10008000000000000000000000000000000070
:1000900000000000808000A000000000000060
:1000A000000000000000000000000000000050
```

In first Iteration, pc module will fetch 4 bytes data at the given address, at 0x80800024, which is 0x80800060. In the second iteration pc module will fetch the data at 0x80800060, which is 0x80800094. In the third iteration, pc module will fetch the data at 0x80800094, which is 0x808000A0. Finally this data will be added to the offset given, i.e. 0x1000 and return the final address, which is 0x808010A0.

GetInfoTableAddress tag is used to locate an address at given offsets. Here we need to specify OFFSET'S to get the final address. The user can specify as many OFFSET'S as he wishes. The final address can be calculated using the API getInfoTableAddress.

```
long getInfoTableAddress(final long startAddress, final List<Integer>
offsetList)
```

In the below example we have take 2 OFFSET'S .

```
List<Integer> offsetList = new List<Integer>();
offsetList.add(4);
offsetList.add(12);
```

```
patchCheckObj.getInfoTableAddress(0x80800020,offsetList);
```

Consider the same sample above hex data.

In the first iteration, pc module adds the start address with the OFFSET1, i.e. 0x80800020 + 0x4, which is 0x80800024, and fetches the data at that location, i.e. 0x80800060. In the second iteration, new address will be added to the OFFSET2, 0x80800060+0x12, which is 0x80800072, and return as the final address.

4.15 Checksum Module

Checksum modules are used to calculate the checksum over the range specified. This is triggered by keyword **CHKSUM**. It also takes the initial and final values for Checksum calculations. Checksum calculation depends on the chosen algorithm. This can be provided by Checksum algorithm names. Possible types of algorithms are {CRC32, ADD32, CRCxx, IFXCRC, ADD16 and ADD8}

This checksum module is loaded by Infoblock module for the checksum over Infoblock and Checksum Infoblock. Default CRC32 checksum algorithm will be used for the Checksum over Infoblock.

For checksum over the checksum blocks, depending on the index in the hex file (checksum Infoblock structure) the algorithm, respective algorithm will be invoked. The index for the selection of the algorithm is as shown below.

INDEX	ALGORITHM
0x0	IFXCRC
0x1	ADD32
0x2	CRC32
0x3	CRC64
0x4	CRC32CCITT
0x5	CRCXX
0x8	ADD8
0x10	ADD16
0x100	MOD256

Checksum algorithm can also be used to calculate the checksum over the hex data ranges and write the checksum result to output hex file or pre-defined address. This is helpful for the user to cross check the checksum results in the hex file.

Checksum module object can be created with the help of function

```
public static IChecksum createCheckSumObject(final Algorithm algorithm, final
IHexDataInterface hexObject)
```

This method returns an object of type IChecksum based on the algorithm specified by the user.

```
private IChecksum checksum = ChecksumFactory.createCheckSumObject(algorithm,
hexDataModel);
```

Checksum Algorithms:

CRC32

```
algorithm = Algorithm.CRC32;
```

```
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

INVCRC can be set using(Used to invert the result)

```
checksum.setInverse(Boolean.parseBoolean(invrc));
```

CRCxx

```
algorithm = Algorithm.CRCxx;  
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

This is similar to CRC32 algorithm, however, Polynomial and its Width is configured by the user in the XML file.

Specifies the Polynomial used for Checksum Algorithm CRCxx. This is required only for CRCxx algorithm.

```
checksum.setInverse(Boolean.parseBoolean(invrc));
```

Specifies the width of the polynomial to be used. It varies from 8 to 32. This is required only for CRCxx algorithm.

```
checksum.setPoly(HexUtils.validateAndConvert(poly));
```

Specifies the width of the polynomial to be used. It varies from 8 to 32. This is required only for CRCxx algorithm

```
checksum.setWidth(Integer.parseInt(width));
```

ADD32

```
algorithm = Algorithm.ADD32;  
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

INVADD32 can be set using(Used to invert the result)

```
checksum.setInverse(Boolean.parseBoolean(invadd32));
```

IFXCRC

```
algorithm = Algorithm.IFXCRC;  
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

ADD16

```
algorithm = Algorithm.ADD16;  
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

INVADD16 can be set using(Used to invert the result)

```
checksum.setInverse(Boolean.parseBoolean(invadd16));
```

ADD 8

```
algorithm = Algorithm.ADD8;
```

```
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

INVADD8 can be set using(Used to invert the result)

```
checksum.setInverse(Boolean.parseBoolean(invadd8));
```

CRC32CCITT

```
algorithm = Algorithm.CRC32CCITT;
```

```
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

CRC64

```
algorithm = Algorithm.CRC64;
```

```
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

INVCRC64 can be set using(Used to invert the result)

```
checksum.setInverse(Boolean.parseBoolean(invcrc64));
```

MOD256

```
algorithm = Algorithm.MOD256;
```

```
checksum = ChecksumFactory.createChecksumObject(algorithm, hexDataModel);
```

Checksum Ranges:

Once the checksum object is obtained, the checksum ranges can be specified using the below APIs.

FROM address can be set using:

Specifies the start address of the checksum range

```
checksum.setFromAddress(long fromAddr);
```

LEN can be set using:

Specifies the length of the checksum range

```
checksum.setLen(long length);
```

STARTVAL can be set using

Specifies the initial value for the checksum algorithm

```
checksum.setStartValue(HexUtils.validateAndConvert(long StartVal));
```

ENDVAL can be set using

Specifies the end value for the checksum algorithm

```
checksum.setEndValue(HexUtils.validateAndConvert(long endval));
```

ALIGN to set align to 4 bytes

```
checksum.setAlignDataTo4Bytes();
```

Compensation Bytes:

This feature is used to calculate the checksum of a block and store the checksum in the hex file. The checksum is written to an address with value 00000000 FFFFFFFF. The address to which the checksum is written does always contain the value 00000000 FFFFFFFF. The value 00000000 is replaced by the checksum value and the value FFFFFFFF is replaced by the value obtained by subtracting the obtained checksum value from FFFFFFFF.

This feature is used only by ADD16 and ADD32 algorithms. This feature is enabled if the value of this attribute is "true". By default the value of this attribute is set to "false" i.e. this feature has to be enabled explicitly.

```
Algorithm add16 = Algorithm.ADD16;
IChecksum add16Checksum = ChecksumFactory.createCheckSumObject(add16, hexObject);
add16Checksum.setCompensationBytes(true);
```

```
Algorithm add32 = Algorithm.ADD32;
IChecksum add32Checksum = ChecksumFactory.createCheckSumObject(add32, hexObject);
add32Checksum.setCompensationBytes(true);
```

4.16 Exception and Logging

HexmodX can be used in three use cases such as running the tool with bean shell script, running the tool with configuration xml files and using it as a dependent library for an application. In all these cases, a standardized exception 'HexmodxException' is being thrown during exception scenarios.

While using it as a single module, the logging mechanism gets enabled. In case of using it as a dependent library, it is disabled by default to avoid logging in the user console and in the log file. It can be enabled if needed.

```
HexmodxLogger.enableConsoleLogging(true);
HexmodxLogger.setLogFilePath("<Log file name>");
Customized exception HexmodxException is created by passing relevant message to the constructor
```

```
public HexmodxException(final String message){
    super(message);
    this.message = new ArrayList<String>();
    this.message.add(message);
}
```

In case of exception occurrence, relevant message is being logged and checking if the objects are of type HexmodxException and throwing the exception object.

```
public static void logAndThrowException(final String tag, final String message, final Exception obj)
throws HexmodxException {
    HexmodxLogger.logMessage(HexmodxLogLevel.ERROR, tag, message);
    if ((obj == null) || !(obj instanceof HexmodxException)) {
        throw new HexmodxException(message);
    }
    throw (HexmodxException) obj;
}
```

When hexmodx is used with Bean shell script and XML configurations, the exception is being handled at the main class, and while using it as dependent library, the tool/module which has dependency on hexmodx handles the exception in the way it is needed.

4.17 Hexcompare

HexCompare module is used to compare the two complete hex files or particular range in two different hex files. HexCompare also generates a report in the text file, which describe the information about the comparison.

To compare two complete hex files:

```
IntelHexFileParser hexFileParser1 = new IntelHexFileParser();
hexFileParser1.setIgnoreLineCks(true);
hexFileParser1.parse(hexObject1, "First hex file name", true);
```

```
IHexDataInterface hexObject2 = HexObjectFactory.createHexObject();
IntelHexFileParser hexFileParser2 = new IntelHexFileParser();
hexFileParser2.setIgnoreLineCks(true);
hexFileParser2.parse(hexObject2, "Second hex file name", true);
```

```
HexCompare hexCompareObject = hexObject1.compare(hexObject2, "result file name");
```

To compare a particular range in two hex files:

```
HexCompare hexCompareObject = hexObject1.compare(hexObject2, Object address, long length,
"result file name");
```

The output of the compare method is a HexCompare object. This object contains 4 lists.

- 1) List of ULBA's present in 1st file only.
- 2) List of ULBA's present in 2nd file only.
- 3) List of ULBA's present in both the file and the data in those ULBA's are same.
- 4) List of ULBA's present in both the file and the data in those ULBA's are different.

Each list contains details about the ULBA address, the two ULBAAddressRecords each from one file, Difference type(Whether that ULBA is present in only 1st file, or 2nd file or present in both the files) And also the list of data differences. (address, no of bytes and difference type i.e if the particular address is a gap in one of the file, or if the dat is different)

The result of the comparison will be stored in the result file specified in the method.

A sample snapshot of the result file is shown below:

```
Address: 80000010
66CF383A9930C7C506554122F9AABEDD
72B7A6B48D48594B100221AEEFFDDE51
```

```
Address: 80000120
10000000C0BA008000C00180B0570080
.....40....B8.....
```

```
Address: 80000190
DD6AC32F22953CD02008910000A7D9AA
B1A213834E5DEC7C.....
```

```
Address: 80000340
FB1C6D00431D8201CDC120090D00C004
1B1D....63.....
```

```
Address: 80000350
6D00151D6D00851D91000028D92226D0
....35.....A5.....
```

.. indicates gaps or the data is same in both the files.

Note: Only addresses in which the data is different is displayed in the result file.

4.18 Cleanup

Cleanup module is used to erase the contents of TPROT (Tuning Protection) and CTPROT (Customer Tuning Protection Block) out from Hex files. The main reason for doing thing is to protect since both sections which are not to be read out.

After removing the TPROT and CTPROT from the hex file, the remaining hex file must be signable, must be good to generate signatures. Therefore it is not sufficient to remove the TPROT and CTPROT totally because in that case the remaining hex file can't be trimmed (checksums) and signed.

```
ICleanup cleanObj = new Cleanup(hexObject);
```

Clean Operation:

Clean operation can be performed by using the API cleanUpDo provided by the ICleanup interface.

```
cleanObj.cleanUpDo(final long startAddress);
```

It cleans only if the blocks are TPROT or CTPROT blocks in hex file.

4.19 Compress Module

Compress module works based on the LZRB algorithm.

The compression module works similar to Checksum module. Currently only LZRB algorithm is implemented.

The compression will be mainly based on FLASH and IMAGE types. FLASH compression types writes the data in particular format in the hex files, while IMAGE type overwrites the hex file data at specified location.

The XML configuration of these options are as shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<CONF xmlns="http://www.rbin.com/mds-1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.rbin.com/mds-1compress.xsd">
  <HEXMOD>
    <COMPRESS >
      <RANGE FROM="0x9000" TO="0x19000" AT="0x00" OUTPUTFILE=".\\TC014\\01\\out\\hexmodtest2.hexf"/>
    </COMPRESS>
  </HEXMOD>
</CONF>
```

Elements	Children or Attributes	Description
COMPRESS	RANGE	Child element of HEXMOD. It initiates the compress operation. Following tags appear under this. RANGE : This supplies range information for compression.
RANGE	FROM TO OUTPUTFILE AT	Range tag has the following attributes. FROM : Start address from where compression begins. If not specified, the default value will be considered as minimum address of the data model. TO : End address from where compression ends. If not specified, the default value will be considered as maximum address of the data model. AT : The address at which it needs to be written to the file. If not specified, the default value will be considered as 0x00. OUTPUTFILE : If the output file type is not specified, the default file type is taken as bin.

4.20 DeleteBlock

DeleteBlock functionality is used to delete the blocks of hex file. Block ID can be specified to delete the block. HexmodX deletes the complete block including the checksum blocks.

Sample HexmodX script to delete the block is,

```
byte infoblockID = Byte.valueOf(0x60, 16);

List<IBlockElement> infoBlocks = this.infoblockManager.getInfoBlocks();

for (IBlockElement iBlockElement : infoBlocks) {
    if (infoblockID == (byte) ((IInfoblock) iBlockElement).getID()) {
        List<IChecksumBlk> checksumBlocks = ((IInfoblock) iBlockElement).getChecksumBlocks();

        for (IChecksumBlk iChecksumBlk : checksumBlocks)
        {
            long adCSStart = ((IMDGChecksumBlk) iChecksumBlk).getAdCSStart();
            long adCSEnd = ((IMDGChecksumBlk) iChecksumBlk).getAdCSEnd();
            this.hexObject.deleteData(adCSStart, (adCSEnd - adCSStart) + 1);
        }
        break;
    }
}
```

4.21 ITRAMS Module

ITRAMS module calculates signature for data part of the ITRAMS hex files and calculates checksum using CRC32 algorithm for header part of that hex files.

Sample HexmodX script to calculate signature for iTrams block,

```
IHexDataInterface hexObject = HexObjectFactory.createHexObject();
IntelHexFileParser hexFileParser = new IntelHexFileParser();
hexFileParser.setIgnoreLineCks(true);
hexFileParser.parse(hexObject, "Mergtput.hex", true);

ItramsMemlayConfiguration itransMemlayConfiguration = new ItramsMemlayConfiguration(hexObject);
itransMemlayConfiguration.parseMemlayFile("iTRAMS_Memlayout.xml", "sigsoo_private.key");

IntelFormatter Formatter = new IntelFormatter();
HexmodXSerializer hexFileSerializer = new HexmodXSerializer(hexObject, Formatter);

hexFileSerializer.serialize("Core_out.hex");
```

4.22 Cluster Linker Module

Cluster linker HEX patching module patches symbol address and value in the cluster HEX files. User need to provide input Hex files, symbol address and its value and final output HEX file

To pass input HEX files first Input files need to be passed in ClusterInputValues

```

this.clusterInputValues.setInputFileName(fileInElement.getAttribute("NAME"));
this.clusterInputValues.setInputFileType(fileInElement.getAttribute("TYPE"));
this.clusterInputValues.setIgLCKS(Boolean.getBoolean(fileInElement.getAttribute("IGNORELINECKS")));
this.clusterInputValues.setOverwrite(Boolean.getBoolean(fileInElement.getAttribute("OVERWRITE")));
this.clusterInputValues.setLittleEndian(isLittleEndian);

```

After setting all the values call parseInputHexFile method in ClusterAddressPatch to parse input HEX files call the below method.

```

@Override
public void parseInputHexFile(final ClusterInputValues clusterInputValues) throws HexmodXException

```

After setting all the values call patchSymbolAddress method in ClusterAddressPatch to patch symbol address and values

```

@Override
public void patchSymbolAddress(final long address, final String value, final String overwrite) throws HexmodXException

```

After setting all the values call createOutputHexFile method in ClusterAddressPatch to create output HEX files call the below method.

```

@Override
public void createOutputHexFile(final String name, final String type, final boolean ignore3or5, final Long layout, final long fillPattern) throws HexmodXException

```